第7章 第5节 Tools 详解

- 来源: https://class.imooc.com/lesson/5280#mid=104934
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第7周 MCP：模型上下文协议
- 模块: Tools、Prompts详解
- 课节: 图文 1-1 Tools 详解

1-1 Tools 详解

第7章 第5节 Tools 详解

> [!TIP] 阅读指南 上一节学习了 Resources 的实战技巧，它让 AI 能够读取结构化的数据。本节学习 MCP 的三大核心能力之一：Tools，让 AI 不仅能读数据，还能执行操作。

5.1 Tools：让 AI 动起来

Resources vs Tools：读与写

上一节学习了 Resources，它是只读的：

只读

但很多时候，AI 需要执行操作：

用户需求的演进：

✓ "让我看看用户表的结构"
   → Resources 可以解决
   → 读取预先准备好的 schema Resource（固定的表结构文档）

× "帮我查询所有注册时间在2024年的用户"
   → Resources 不适合做
   → 原因：这需要"动态查询"数据库
   → 虽然是只读，但数据是"实时生成"的，不是预先存在的文件
   → 解决：需要 Tool 来执行 SQL 查询

× "帮我发送一封感谢邮件给所有VIP用户"
   → Resources 不适合做
   → 这是写操作，有副作用

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
```

预览

> 补充说明：Resources vs Tools 的"只读"区别 Resources 的"只读"：读取预先存在的固定数据 这就像读取一个文件（如 schema.json 或 sample_data.json），数据本身是静态的，不需要在读取时进行额外的逻辑计算。 Tools 的"查询"：执行动态操作生成数据 例如执行 SQL 语句 SELECT * FROM users WHERE ...，虽然这可能也是"只读"的操作，但数据是根据查询条件"动态计算"出来的。每次查询条件不同，返回的结果就会发生变化。

这就是 Tools 要解决的问题。

Tools 的核心特征

Tools 是 MCP 中赋予 AI 「行动力」的核心组件。与静态的 Resources 不同，它们本质上是可执行的函数，能够执行动态操作，并可能产生副作用（如写入数据库或调用外部 API）。在实际交互中，由 LLM 根据用户意图自主决定调用哪个 Tool 以及传递具体的参数。

常见的 Tools 示例包括： • 搜索文档内容 (search_docs tool) • 执行数据库查询 (query_database tool) • 调用 API (call_API tool) • 发送邮件 (send_email tool)

维度	Resources	Tools
本质	数据源	函数
操作	只读	可读可写
内容	静态信息	动态执行
选择方式	LLM 看 description	LLM 看 description + parameters
副作用	无	可能有

如何判断用 Resources 还是 Tools

这是很多人的困惑：为什么有些场景似乎两者都能用?

为什么有些场景似乎两者都能用?

核心判断标准：数据是「静态元数据」还是「动态业务数据」

这种区分并不是看是否需要调用函数（因为两者内部都可以调用函数），而是看查询的数据性质：

• Resources 适用于那些相对固定、可枚举的元数据（如表结构、系统配置、静态文档）。它就像是一张「地图」，告诉 AI 这里的地形如何。 • Tools 适用于那些频繁变化、需要实时计算的业务数据（如实时库存、用户信息查询、发送邮件）。它就像是一把「瑞士军刀」，让 AI 能根据当前情况采取行动。

可以通过几个典型场景来对比：

场景	推荐方案	理由
查看用户表的字段定义	Resource	表结构是相对固定的「元数据」
查询 2024 年注册的用户	Tool	业务数据查询，结果随参数动态生成
获取上海此时此刻的天气	Tool	实时数据，且涉及外部 API 调用
查阅系统的 API 认证文档	Resource	静态文档，内容预先存在

原则 如果数据是「固定的、用来描述系统的（Schema、文档）」，用 Resource；如果数据是「动态的、属于业务逻辑的（订单、天气、操作）」，用 Tool。

Tools 工作流程

上一节用 Resources 让 AI 读取静态的API文档，现在用 Tools 让 AI 执行具体操作（比如测试接口）。

```
场景1：测试登录接口（用 Tools）
═══════════════════════
用户："帮我测试一下登录接口，用户名 test，密码 123456"
  ↓
1. LLM 看到 Tools 列表
2. 选择 "test_api_endpoint" Tool
3. 传参数：{"endpoint": "/auth/login", "method": "POST", "data": {"username": "test",
"password": "123456"}}
4. MCP Server 执行 API 调用
5. 返回结果："返回 200 OK，token: eyJhbGc..."
→ 有副作用：调用了外部API，可能创建了session
```

完整流程图

5.2 Tools 的协同与安全

Resource + Tool 组合使用的威力

在真实的项目中，Resources 和 Tools 往往是形影不离的。Resources 负责提供背景知识（Context），而 Tools 负责执行具体动作（Action）。

示例：智能数据库助手（伪代码）

```
class DatabaseMCPServer:
    def list_resources(self):
        # ▸ 暴露表结构（Resource）
        return [{"uri": "db://schema/orders", "description": "订单表结构"}]

    def list_tools(self):
        # ▸ 提供查询能力（Tool）
        return [{
            "name": "query_sql",
            "description": "执行 SQL 查询",
            "inputSchema": {"type": "object", "properties": {"sql": {"type": "string"}}}
        }]

    def call_tool(self, name, args):
        # 执行查询逻辑...
        pass
```

这种组合的威力在于：当用户问「分析 2024 年订单」时，AI 会先去读取 Resource 搞清楚 orders 表有哪些字段，然后基于这些知识，精准地构造出 SQL 语句去调用 Tool。这种「先看说明书，再上手操作」的模式，极大地提高了 AI 的准确度。

Tool 的权限控制

由于 Tools 具有「副作用」，权限控制就成了系统的护城河。

> [!NOTE] 「副作用」是什么意思？ 在编程里，一个函数如果只是读取数据、计算并返回结果，它不会改变外部世界的任何东西——这叫「没有副作用」。但如果一个函数写入了数据库、发送了邮件、删除了文件、调用了外部 API……它就在改变外部世界的状态，这就叫「有副作用」。 打个比方：去图书馆查一本书（读操作）不会改变图书馆的藏书；但如果在书上写字、把书撕掉一页、或者把书寄给朋友（写操作），图书馆就被改变了。Tools 能做后者——所以必须给它配上严格的权限关卡。

这种分层防御确保了 AI 在获得行动力的同时，不会变成失控的洪水猛兽。通过一段精简的逻辑来观察这种「安全关卡」是如何工作的：

```
def call_tool(name, arguments):
    # ▸ 1. 语法拦截：检查并过滤高危 SQL（如 DROP/DELETE）
    if is_dangerous(arguments.get("sql", "")):
        return "拒绝执行：检测到高危指令"

    # ▸ 2. 身份校验：验证当前用户是否有权访问目标资源
    if not has_access(user_identity, arguments.get("table")):
        return "拒绝执行：权限不足"

    # ▸ 3. 业务执行：双重检查通过后，才执行真正的逻辑
    return perform_logic(name, arguments)
```

可以看到，在执行Tools函数时，可以进行各种检测，只有安全的操作才能被执行。

5.3 下一节预告

Tools 让 AI 拥有了执行操作的能力，但用户的每次需求都需要"从零开始"描述。下一节将介绍 MCP 的第三大核心能力——Prompts（预制的使用模板），它能把常见的操作流程标准化，让用户不再重复描述需求，也让 AI 的输出更加一致可控。

5.4 ■ 学点英语

中文	English	音标	说明
读写分离	Read-Write Separation	/riːd raɪt ˌsepəˈreɪʃn/	Resources只读与Tools可执行的职责划分设计
三层检查	Three-Layer Check	/θriː ˈleɪər tʃek/	语法拦截→身份校验→业务执行的Tool安全检查架构
参数校验	Input Validation	/ˈɪnpʊt ˌvælɪˈdeɪʃn/	对Tool参数进行类型、范围和格式的合法性检查
副作用操作	Side Effect Operation	/saɪd ɪˈfekt ˌɑːpəˈreɪʃn/	有状态变更或外部影响的写操作，需额外安全控制

图片

